

C360-Mesh

Preliminary Research and Architecture Rationale

Inter IIT Tech Meet 15.0 Prepathon - Natural Language Processing

Research objective. Identify production-oriented patterns for ambient customer agents, memory, streaming freshness, multi-agent coordination, intervention selection and HITL, then convert those findings into explicit architectural decisions. The focus is not to collect references for their own sake, but to show how each source changes the proposed system.

1. What the problem actually demands

The central output is a decision, not a dashboard. For each evolving customer timeline, the system must choose a concrete intervention from a bounded set or explicitly choose no action, and preserve enough evidence to explain and evaluate that choice. This shifts the design from “retrieve context and generate text” to an event-driven decision system with memory, action constraints, and auditable state transitions.

- Continuous multi-source ingestion: usage, support/communication, transactions/billing, KYC/compliance and related signals arrive asynchronously.
- Persistent per-customer memory: recent working context, customer-specific episodic history and shared semantic/policy knowledge must remain distinct.
- Multi-agent specialization: agents should have different data access, tools and roles; structured handoffs matter more than agent count.
- Trust boundary: deterministic guardrails, PII minimization, safe tool permissions and real HITL are part of the architecture, not documentation-only claims.
- Evaluation-ready outputs: timestamped inferred customer state, confidence and action/no-action must be emitted in a reproducible schema.

2. Multi-agent coordination: why a state board instead of a chat room

Research finding. Cemri et al. (2025) analyze five popular multi-agent frameworks across more than 150 tasks and identify 14 recurring failure modes, grouped into system/specification failures, inter-agent misalignment, and verification/termination failures. The result argues against assuming that adding more agents automatically improves reliability.

Architectural consequence. C360-Mesh therefore treats agent communication as typed state updates. Specialist agents publish Evidence Contracts to a shared per-customer board: claim, value, confidence, source event IDs, observation time, validity and model version. The Synthesis Agent consumes those conclusions; it does not ingest unrestricted “thought transcripts” from upstream agents.

Adaptive verification. Debate/verification is activated only when structured findings materially disagree, confidence is below a threshold, or the proposed action crosses a risk boundary. This preserves the benefit of multiple perspectives without paying the cost and instability of debate on every event.

3. Memory: from static retrieval to a temporal customer digital twin

Research finding. Rasmussen et al. (2025) describe Zep/Graphiti as a temporally aware knowledge-graph memory layer that combines structured business data and unstructured interaction data while preserving historical relationships. The important idea for this problem is not that a graph database is mandatory; it is that facts change over time and the old state often remains important for reasoning and audit.

Architectural consequence. C360-Mesh separates working, episodic and semantic memory and adds Temporal Truth Maintenance. Each important state fact can carry `valid_from`, `valid_until`, evidence provenance and `supersedes/superseded_by` links. If a late support event changes churn risk from LOW to HIGH, the old fact is not deleted; it is closed and linked to the revised fact. This supports “what did the system know at time T?” and makes post-mortems reproducible.

- Working memory: current open case, newest findings, unresolved conflicts and pending actions.
- Episodic memory: past incidents, proposed interventions, human approvals/overrides and observed outcomes for this customer.
- Semantic/shared memory: policy, offer eligibility, general risk patterns and compliance knowledge.
- Life-event memory: persistent hypotheses (for example relocation or home purchase) with confidence, supporting/contradicting evidence and temporal decay.

4. Streaming correctness: event time, lateness and replayability

Research finding. Apache Flink distinguishes event time (when the event actually happened) from processing/arrival time and uses watermarks to reason about progress in out-of-order streams. Its windowing model also makes lateness an explicit design decision rather than silently assuming ordered data.

Architectural consequence. Every normalized event in C360-Mesh carries `event_time` and `ingestion_time` plus a unique event ID. The ingestion layer deduplicates, keys events by customer/account, tracks bounded lateness and recomputes affected aggregates when late events are still within the retention window. A replayable append-only event log enables deterministic scenario evaluation and debugging.

Implementation choice for the competition. The architecture borrows streaming semantics without requiring a heavyweight Flink/Kafka cluster for the first prototype. Redis Streams is suitable as a lightweight append-only stream with consumer-group delivery, while PostgreSQL remains the system of record. If final scale requires it, the same event contract can be placed behind Kafka/Flink without changing downstream agents.

5. Intervention selection: prediction is not action

Research finding. Uplift modeling explicitly distinguishes “who is likely to churn” from “whose behavior is likely to change because of a treatment.” Moraes et al. (2023) frame uplift modeling as treatment-effect estimation for personalization and note the need to weigh promotional benefit against cost.

Architectural consequence. The Synthesis Agent does not directly invent an action. It emits a grounded customer state and reason codes. A bounded action generator then enumerates allowed candidates, including `NO_ACTION`. A transparent policy/utility layer scores candidates using estimated benefit, direct cost, eligibility/policy constraints, risk and uncertainty. This gives the system a defensible reason for choosing an RM call over a large retention credit, or for choosing no intervention at all.

Research limitation stated explicitly. Without randomized treatment/outcome history, the prototype must not claim accurate individual causal effects. Initial utility estimates will be rule- and history-informed. If suitable labeled intervention data later becomes available, the same interface can be upgraded to uplift trees, causal forests, or a constrained contextual policy.

6. Human-in-the-loop, safety and decision fragility

Research finding. LangGraph interrupts and persistence demonstrate a useful implementation pattern for genuine HITL: workflow state is checkpointed, execution pauses, and an external reviewer can approve, edit or reject before the graph resumes. This is materially different from merely logging “human approved.”

Architectural consequence. C360-Mesh computes a risk/ambiguity level from impact, uncertainty, sensitivity and irreversibility. Low-risk actions can proceed within a bounded tool permission; uncertain or high-impact actions pause at HITL; deterministic hard-stop conditions route directly to specialist escalation.

Evidence-ablation stability check. Before high-impact actions, the system removes the strongest supporting evidence and recomputes the recommendation. If the selected action changes sharply, the decision is labeled fragile and escalated. This is a robustness heuristic, not a claim of causal counterfactual inference. Its purpose is to turn explainability into an operational safety signal rather than a post-hoc paragraph.

7. Retrieval and storage choices

PostgreSQL + pgvector. pgvector supports exact nearest-neighbor search as well as HNSW and IVFFlat approximate indexes. For a competition-scale system, keeping customer state, audit records, interventions and vector retrieval in PostgreSQL reduces operational complexity and makes the repository easier to reproduce. A dedicated vector database can be introduced later if load tests justify it.

Retrieval policy. Semantic similarity alone is insufficient for customer memory. Retrieval should filter by customer/account scope first and then combine semantic similarity with temporal recency, evidence reliability and current-state relevance. This prevents semantically similar but stale or cross-customer records from dominating the synthesis context.

8. Final preliminary architecture decisions

Design area	Frozen preliminary decision
Orchestration	State-centric workflow with typed agent I/O; targeted verifier/debate only under disagreement or low confidence.
Streaming	Schema adapter + event log + event-time/lateness semantics; lightweight Redis Streams prototype, swappable with heavier infrastructure.
Memory	Working + episodic + semantic stores plus temporal validity/supersession; customer/account isolation enforced in storage queries.
Specialists	Statistics/ML for numerical signals, NLP for support text, deterministic rules for guardrails; LLM reserved mainly for synthesis/ambiguity.
Decision	Bounded action set with explicit NO_ACTION; utility/eligibility/risk layer constrains action selection.
HITL	Real pause/resume checkpoint with approve/modify/reject and full context snapshot.
Audit	One decision trace links event IDs, evidence, retrievals, model/tool calls, final action, and human

	override/outcome.
Evaluation	Replay adversarial scenarios including late events, disagreement, stale memory, guardrail triggers and customer-isolation tests.

9. Research log: source -> learning -> design impact

Source	Key learning	Decision informed
Problem Statement: Agentic Customer 360 - Proactive Intervention Desk	Defines bounded actions/no-action, three-layer memory inspiration, shared state board, multiple coordination patterns, live stream/HITL/guardrail/traceability production bar and evaluation outputs.	Sets non-negotiable requirements and prevents solving the task as a generic chatbot.
Cemri et al. (2025), Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657	Identifies 14 MAS failure modes across system design, inter-agent alignment and verification/termination.	Use typed evidence contracts, scoped state, explicit verification and fewer dynamically activated agents.
Rasmussen et al. (2025), Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956	Shows a temporal memory architecture that synthesizes structured and unstructured information while preserving historical relationships.	Use temporal validity/supersession and persistent life-event memory; graph ideas inform design even if prototype storage is relational.
Apache Flink, Timely Stream Processing / Event Time and Watermarks	Event time and watermarks provide consistent reasoning for out-of-order and late streams; windows make lateness policy explicit.	Store event_time and ingestion_time; implement bounded lateness, recomputation and replay.
Redis, Streaming documentation	Redis Streams provides an append-only time-ordered log with consumer groups, acknowledgements and reassignment.	Use as lightweight event transport in prototype without coupling downstream logic to Redis.
LangGraph, Interrupts and Persistence documentation	Workflow state can be checkpointed, paused for external input and resumed; supports approve/edit/reject HITL patterns.	Implement a real approval checkpoint instead of a simulated log message.
Moraes et al. (2023), Uplift Modeling: from Causal Inference to Personalization. arXiv:2308.09066	Treatment-effect viewpoint targets incremental intervention impact and considers benefit versus cost.	Use a bounded utility layer and distinguish churn prediction from intervention effectiveness; avoid unsupported causal claims without treatment data.
pgvector project documentation	PostgreSQL can provide exact vector search and HNSW/IVFFlat approximate indexes.	Keep structured state, audit data and semantic retrieval in one reproducible database for the prototype.

10. Known limitations and near-term validation plan

- Organizer event schema and hidden ground truth are not available in the PS itself. A schema adapter isolates this uncertainty so the downstream system is not coupled to guessed field names.

- True uplift/treatment-effect estimation requires appropriate intervention/outcome data. The first version uses transparent priors/history and records outcomes for later learning.
- Life-event inference can be sensitive and uncertain. It is stored as a hypothesis with confidence and evidence, never as a definitive personal fact, and customer-facing use is subject to HITL/policy checks.
- The first prototype prioritizes correctness and replayability over infrastructure scale. Distributed replacements can be introduced behind stable event/state interfaces.
- Validation will include deliberate failure cases: late/out-of-order events, duplicate events, conflicting agents, stale memories, cross-customer leakage attempts, legal/fraud hard stops, and fragile high-impact recommendations.

References

- [1] M. Cemri et al., “Why Do Multi-Agent LLM Systems Fail?”, arXiv:2503.13657, 2025. <https://arxiv.org/abs/2503.13657>
- [2] P. Rasmussen et al., “Zep: A Temporal Knowledge Graph Architecture for Agent Memory”, arXiv:2501.13956, 2025. <https://arxiv.org/abs/2501.13956>
- [3] Apache Flink Documentation, “Timely Stream Processing: Event Time and Watermarks.” <https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/>
- [4] Redis Documentation, “Redis Streaming / Streams.” <https://redis.io/docs/latest/develop/use-cases/streaming/>
- [5] LangChain Documentation, “LangGraph Interrupts” and “Persistence.” <https://docs.langchain.com/oss/python/langgraph/interrupts>
- [6] F. Moraes et al., “Uplift Modeling: from Causal Inference to Personalization”, arXiv:2308.09066, 2023. <https://arxiv.org/abs/2308.09066>
- [7] pgvector, “Open-source vector similarity search for Postgres.” <https://github.com/pgvector/pgvector>